

FRD

Functional Requirements Document (FRD)

AWS Glue Data Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based data pipeline that ingests customer and order data from S3, performs data quality operations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregates for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- SCD Type 2 implementation for order summary tracking
- Incremental processing logic triggered by customer updates
- Aggregation of customer spending metrics
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Infrastructure provisioning or IAM policies
- Scheduling or orchestration mechanisms
- Data retention policies
- Disaster recovery procedures

Assumptions

1. CSV files in the source S3 paths are well-formed and use standard delimiters (comma-separated)
2. The term "Null" (string) and NULL (actual null values) both represent missing data
3. Duplicate detection is based on all columns in each dataset
4. SCD2 "change" detection compares all customer and order attributes
5. The Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` already exists
6. Hudi table format is used for SCD2 implementation with default Hudi configurations

7. "Incremental logic" means the pipeline processes only new or changed customer records after initial load
8. Date fields in Order data are in a parseable format (e.g., YYYY-MM-DD or MM/DD/YYYY)
9. AWS Glue jobs have sufficient permissions to read from source buckets and write to target buckets
10. Athena is used for querying the final tables but query definitions are out of scope
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3
- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a DataFrame for processing.

- Business Objective:

Enable the pipeline to access customer master data for downstream processing and SCD2 operations.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Format: CSV
 - Schema:
 - CustId (data type not specified in BRD)
 - Name (data type not specified in BRD)
 - EmailId (data type not specified in BRD)
 - Region (data type not specified in BRD)
- Processing Rules / Functional Steps:
 1. Connect to S3 path `s3://adif-sdlc/sdlc\_wizard/customerdata/`
 2. Read all CSV files in the path
 3. Parse CSV with header row
 4. Load data into a structured DataFrame
- Outputs:
 - In-memory customer DataFrame with columns: CustId, Name, EmailId, Region
- Preconditions:
 - S3 bucket and path exist and are accessible

- CSV files contain header row matching the schema
- Postconditions:
- Customer data is available in memory for cleaning and processing
- Error / Validation Conditions:
- If S3 path does not exist, raise error and halt pipeline
- If CSV files are malformed or unreadable, log error and skip file
- If required columns are missing, raise schema validation error
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3

- Description:

The system shall read order transaction data in CSV format from the specified S3 location and load it into a DataFrame for processing.

- Business Objective:

Enable the pipeline to access order transaction data for joining with customer data and generating order summaries.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId (data type not specified in BRD)
- ItemName (data type not specified in BRD)
- PricePerUnit (data type not specified in BRD)
- Qty (data type not specified in BRD)
- Date (data type not specified in BRD)
- CustId (data type not specified in BRD)
- Processing Rules / Functional Steps:
- 1. Connect to S3 path `s3://adif-sdlc/sdlc\_wizard/orderdata/`
- 2. Read all CSV files in the path
- 3. Parse CSV with header row
- 4. Load data into a structured DataFrame

- Outputs:
- In-memory order DataFrame with columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Preconditions:
- S3 bucket and path exist and are accessible
- CSV files contain header row matching the schema
- Postconditions:
- Order data is available in memory for cleaning and processing
- Error / Validation Conditions:
- If S3 path does not exist, raise error and halt pipeline
- If CSV files are malformed or unreadable, log error and skip file
- If required columns are missing, raise schema validation error
-

FR-CLEAN-001: Remove Null and Duplicate Values from Customer Data

- Title: Clean Customer Dataset
- Description:

The system shall remove rows containing null values (both string "Null" and actual NULL) and eliminate duplicate records from the customer dataset.

- Business Objective:

Ensure data quality by removing incomplete and redundant customer records before downstream processing.

- Inputs:
- In-memory customer DataFrame from FR-INGEST-001
- Processing Rules / Functional Steps:
 1. Identify rows where any column contains the string "Null" (case-sensitive) or actual NULL value
 2. Remove all such rows from the DataFrame
 3. Identify duplicate rows based on all columns (CustId, Name, EmailId, Region)
 4. Retain only the first occurrence of each duplicate set
 5. Remove all subsequent duplicates
- Outputs:
- Cleaned customer DataFrame with no null values and no duplicates
- Preconditions:

- Customer DataFrame is loaded in memory
- Postconditions:
- Customer DataFrame contains only complete, unique records
- Error / Validation Conditions:
- If all rows are removed due to null values, log warning and continue with empty dataset
- If no duplicates are found, proceed without modification
-

FR-CLEAN-002: Remove Null and Duplicate Values from Order Data

- Title: Clean Order Dataset
- Description:

The system shall remove rows containing null values (both string "Null" and actual NULL) and eliminate duplicate records from the order dataset.

- Business Objective:

Ensure data quality by removing incomplete and redundant order records before downstream processing.

- Inputs:
- In-memory order DataFrame from FR-INGEST-002
- Processing Rules / Functional Steps:
 1. Identify rows where any column contains the string "Null" (case-sensitive) or actual NULL value
 2. Remove all such rows from the DataFrame
 3. Identify duplicate rows based on all columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)
 4. Retain only the first occurrence of each duplicate set
 5. Remove all subsequent duplicates
- Outputs:
- Cleaned order DataFrame with no null values and no duplicates
- Preconditions:
- Order DataFrame is loaded in memory
- Postconditions:
- Order DataFrame contains only complete, unique records
- Error / Validation Conditions:

- If all rows are removed due to null values, log warning and continue with empty dataset
- If no duplicates are found, proceed without modification
-

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Customer Data
- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog for query access and metadata management.

- Business Objective:

Enable SQL-based querying of customer data via Athena and maintain centralized metadata.

- Inputs:
 - Cleaned customer DataFrame from FR-CLEAN-001
 - Database Name: `gen_ai_poc_databrickscoe`
 - Table Name: `sdhc_wizard_customer`
- Processing Rules / Functional Steps:
 1. Write cleaned customer DataFrame to S3 in a queryable format (e.g., Parquet)
 2. Register the S3 location as a table in Glue Data Catalog
 3. Use database name: ``gen_ai_poc_databrickscoe``
 4. Use table name: ``sdhc_wizard_customer``
 5. Infer or define schema based on DataFrame structure
- Outputs:
 - Glue Catalog table: ``gen_ai_poc_databrickscoe.sdhc_wizard_customer``
 - Customer data stored in S3 (location not specified in BRD)
- Preconditions:
 - Glue Data Catalog database ``gen_ai_poc_databrickscoe`` exists
 - Cleaned customer DataFrame is available
- Postconditions:
 - Customer table is queryable via Athena
 - Table metadata is registered in Glue Catalog
- Error / Validation Conditions:
 - If database does not exist, raise error and halt

- If table already exists, handle based on configuration (overwrite or append)
-

FR-CATALOG-002: Register Order Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Order Data
- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog for query access and metadata management.

- Business Objective:

Enable SQL-based querying of order data via Athena and maintain centralized metadata.

- Inputs:
 - Cleaned order DataFrame from FR-CLEAN-002
 - Database Name: gen_ai_poc_databrickscoe
 - Table Name: sdlc_wizard_order
- Processing Rules / Functional Steps:
 1. Write cleaned order DataFrame to S3 in a queryable format (e.g., Parquet)
 2. Register the S3 location as a table in Glue Data Catalog
 3. Use database name: `gen\_ai\_poc\_databrickscoe`
 4. Use table name: `sdlc\_wizard\_order`
 5. Infer or define schema based on DataFrame structure
- Outputs:
 - Glue Catalog table: `gen\_ai\_poc\_databrickscoe.sdlc\_wizard\_order`
 - Order data stored in S3 (location not specified in BRD)
- Preconditions:
 - Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` exists
 - Cleaned order DataFrame is available
- Postconditions:
 - Order table is queryable via Athena
 - Table metadata is registered in Glue Catalog
- Error / Validation Conditions:
 - If database does not exist, raise error and halt
 - If table already exists, handle based on configuration (overwrite or append)

-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Create and Maintain Order Summary with SCD Type 2 Logic

- Description:

The system shall join customer and order data, apply Slowly Changing Dimension Type 2 logic to track historical changes, and store the result in a Hudi table on S3.

- Business Objective:

Maintain a complete historical record of order summaries, capturing changes to customer or order attributes over time while preserving the ability to query data as of any point in time.

- Inputs:

- Cleaned customer DataFrame from FR-CLEAN-001
- Cleaned order DataFrame from FR-CLEAN-002
- Target Table: ordersummary
- Target Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Database Name: gen_ai_poc_databrickscoe
- Table Name: ordersummary

- Processing Rules / Functional Steps:

1. Join customer and order DataFrames on CustId
2. For each joined record, check if it exists in the target `ordersummary` table
3. If the record is new (does not exist):

- Insert as a new row with IsActive = true, StartDate = current timestamp, EndDate = null, OpTs = current timestamp

4. If the record exists and attributes have changed:

- Update the existing row: set IsActive = false, EndDate = current timestamp, OpTs = current timestamp

- Insert a new row with updated attributes: IsActive = true, StartDate = current timestamp, EndDate = null, OpTs = current timestamp

5. If the record exists and attributes have not changed:

- No action required

6. Write the resulting dataset to S3 using Hudi table format

7. Register or update the table in Glue Data Catalog

- Outputs:

- Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Glue Catalog table: `gen\_ai\_poc\_databricksco.ordersummary`
- Schema includes: all customer fields, all order fields, IsActive (boolean), StartDate (timestamp), EndDate (timestamp), OpTs (timestamp)
- Preconditions:
 - Cleaned customer and order DataFrames are available
 - Target S3 location is accessible
 - Hudi libraries are available in Glue environment
- Postconditions:
 - Order summary table contains current and historical records
 - Active records have IsActive = true and EndDate = null
 - Closed records have IsActive = false and EndDate populated
- Error / Validation Conditions:
 - If join results in no records, log warning and create empty table
 - If Hudi write fails, raise error and halt pipeline
 - If duplicate active records are detected for the same business key, raise data integrity error
-

FR-INCR-001: Process Incremental Customer Updates

- Title: Trigger SCD2 Updates Based on Customer Changes
- Description:

The system shall detect new or changed customer records and trigger SCD Type 2 updates in the order summary table to reflect those changes.

- Business Objective:

Ensure that changes to customer master data are propagated to the order summary table incrementally, avoiding full reprocessing of all data.

- Inputs:
 - New or updated customer records from s3://adif-sdlc/sdlc_wizard/customerdata/
 - Existing `ordersummary` table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Processing Rules / Functional Steps:
 1. Identify customer records that are new or have changed since the last pipeline run
 2. For each changed customer, retrieve all associated order records from the order dataset
 3. Apply SCD Type 2 logic as defined in FR-SCD2-001 for the affected records
 4. Update the `ordersummary` table incrementally (close old rows, insert new active rows)

5. Update OpTs (operation timestamp) for all modified rows

- Outputs:
- Updated `ordersummary` Hudi table with incremental changes applied
- Preconditions:
- Initial load of `ordersummary` table is complete (FR-SCD2-001)
- Mechanism exists to identify new/changed customer records (e.g., watermark, change data capture)
- Postconditions:
- Order summary table reflects the latest customer data
- Historical records are preserved with appropriate IsActive, StartDate, EndDate values
- Error / Validation Conditions:
- If no customer changes are detected, skip processing and log informational message
- If incremental update fails, raise error and halt pipeline
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Aggregate Spend Analytics Table
- Description:

The system shall aggregate order data by customer to calculate total spending and store the result in an analytics table for business intelligence purposes.

- Business Objective:

Provide a summarized view of customer spending to support reporting, dashboards, and business analysis.

- Inputs:
- Active records from `ordersummary` table (IsActive = true)
- Target Table: customeraggregatespend
- Target Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Database Name: gen_ai_poc_databricksco
- Table Name: customeraggregatespend
- Processing Rules / Functional Steps:
- 1. Filter `ordersummary` table for active records (IsActive = true)
- 2. Calculate TotalAmount for each order as: PricePerUnit Qty

3. Group by customer Name and Date
4. Sum TotalAmount for each group
5. Select columns: Name, TotalAmount, Date
6. Write the aggregated dataset to S3 at the specified location
7. Register or update the table in Glue Data Catalog

- Outputs:
- Analytics table at s3://adif-sdlc/analytics/customeraggregatespend/
- Glue Catalog table: `gen\_ai\_poc\_databrickscoe.customeraggregatespend`
- Schema:
- Name (data type not specified in BRD)
- TotalAmount (data type not specified in BRD)
- Date (data type not specified in BRD)
- Preconditions:
- `ordersummary` table exists and contains data
- Target S3 location is accessible
- Postconditions:
- Customer aggregate spend table is available for querying via Athena
- Table contains one row per customer per date with total spending
- Error / Validation Conditions:
- If `ordersummary` table is empty, create empty aggregate table and log warning
- If aggregation fails due to data type issues (e.g., non-numeric PricePerUnit or Qty), raise error and halt
-

FR-CATALOG-003: Register Order Summary Table in Glue Data Catalog

- Title: Catalog Registration for Order Summary Hudi Table
- Description:

The system shall register the `ordersummary` Hudi table in the AWS Glue Data Catalog to enable querying via Athena.

- Business Objective:

Provide SQL query access to the SCD Type 2 order summary table for analytical and operational use cases.

- Inputs:

- `ordersummary` Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Database Name: gen_ai_poc_databricksco
- Table Name: ordersummary
- Processing Rules / Functional Steps:
 1. Register the Hudi table location in Glue Data Catalog
 2. Use database name: `gen\_ai\_poc\_databricksco`
 3. Use table name: `ordersummary`
 4. Define schema including SCD2 metadata columns: IsActive, StartDate, EndDate, OpTs
- Outputs:
 - Glue Catalog table: `gen\_ai\_poc\_databricksco.ordersummary`
- Preconditions:
 - `ordersummary` Hudi table exists at the specified S3 location
 - Glue Data Catalog database exists
- Postconditions:
 - Order summary table is queryable via Athena with Hudi support
- Error / Validation Conditions:
 - If table registration fails, raise error and halt pipeline
-

FR-CATALOG-004: Register Customer Aggregate Spend Table in Glue Data Catalog

- Title: Catalog Registration for Customer Aggregate Spend Table
- Description:

The system shall register the `customeraggregatespend` table in the AWS Glue Data Catalog to enable querying via Athena.

- Business Objective:

Provide SQL query access to customer spending aggregates for reporting and analytics.

- Inputs:
 - `customeraggregatespend` table at s3://adif-sdlc/analytics/customeraggregatespend/
 - Database Name: gen_ai_poc_databricksco
 - Table Name: customeraggregatespend
- Processing Rules / Functional Steps:

1. Register the table location in Glue Data Catalog
2. Use database name: `gen\_ai\_poc\_databrickscoe`
3. Use table name: `customeraggregatespend`
4. Define schema with columns: Name, TotalAmount, Date

- Outputs:

- Glue Catalog table: `gen\_ai\_poc\_databrickscoe.customeraggregatespend`

- Preconditions:

- `customeraggregatespend` table exists at the specified S3 location
- Glue Data Catalog database exists

- Postconditions:

- Customer aggregate spend table is queryable via Athena

- Error / Validation Conditions:

- If table registration fails, raise error and halt pipeline

-

3. Non-Functional Considerations

Monitoring and Observability

- AWS CloudWatch shall be used to monitor Glue job execution, log errors, and track performance metrics
- CloudWatch logs shall capture detailed execution logs for troubleshooting

Data Governance (Optional)

- AWS Lake Formation may be used to manage access control, data permissions, and governance policies for the Glue Data Catalog and S3 data lakes

Storage and Compute

- Amazon S3 shall be used for all data storage (source, curated, analytics)
- AWS Glue (Apache Spark) shall be used for all ETL processing
- Apache Hudi shall be used for SCD Type 2 table format

Query Engine

- Amazon Athena shall be used for ad-hoc querying of Glue Catalog tables

-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Description	
-----	-----	-----	
FR-INGEST-001	Source Input (AWS S3), Schemas	Customer CSV ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/	
FR-INGEST-002	Source Input (AWS S3), Schemas	Order CSV ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/	
FR-CLEAN-001	Data Cleaning	Remove "Null" and NULL values, remove duplicates from customer data	
FR-CLEAN-002	Data Cleaning	Remove "Null" and NULL values, remove duplicates from order data	
FR-CATALOG-001	Glue Catalog Tables	Register sdlc_wizard_customer in gen_ai_poc_databrickscoe	
FR-CATALOG-002	Glue Catalog Tables	Register sdlc_wizard_order in gen_ai_poc_databrickscoe	
FR-SCD2-001	SCD Type 2 (Hudi on S3)	Implement SCD2 for ordersummary table with join logic	
FR-INCR-001	Incremental Logic	Process customer updates and trigger SCD2 updates	
FR-AGG-001	Aggregation Table	Generate customeraggregatespend with Name, TotalAmount, Date	
FR-CATALOG-003	Glue Catalog Tables	Register ordersummary table in catalog	
FR-CATALOG-004	Glue Catalog Tables	Register customeraggregatespend table in catalog	

•

• End of Functional Requirements Document